

MindAI Field Manual

A complete walkthrough of a retrieval system that answers questions about any GitHub repository with line-level citations — and refuses to answer rather than invent one. Every design decision, every alternative rejected, and the numbers measured on the running system.

Stack LangGraph · FastAPI · React · Qdrant Models OpenAI · Cohere Languages parsed 15

§ 01

The core idea

Ask a question about a codebase in plain English. Get an answer where every claim points at a real file and line number — or get told the system doesn't know.

General chatbots answer questions about code badly for one specific reason: they have never seen *your* repository. They reply from a blurry memory of similar code on the internet, and the answer looks right while naming functions that do not exist. For a developer trying to understand an unfamiliar codebase, a confident wrong answer is worse than no answer, because verifying it costs more than reading the code would have.

MindAI fixes this by never letting the model answer from memory. The repository is cloned, cut into meaningful pieces, and stored in a searchable index. When a question arrives, the system finds the pieces most likely to contain the answer and hands *only those* to the model, with an instruction: answer using this material and cite the file and line for every claim. If nothing relevant is found, or the model cannot produce a citation, the system refuses.

The 30-second version

“It's a Graph-RAG system for codebases. It clones a repo, uses tree-sitter to split the source at function and class boundaries instead of arbitrary token windows, and indexes each chunk twice — a dense vector for meaning and a sparse BM25 vector for exact keywords. At query time it searches both, fuses the rankings,

reranks with a cross-encoder, and for questions about how code flows it expands to callers and callees using a call graph built at ingest. Answers must cite `path: line` , and low confidence triggers a refusal instead of a guess.”

WHY THIS MATTERS IN AN INTERVIEW

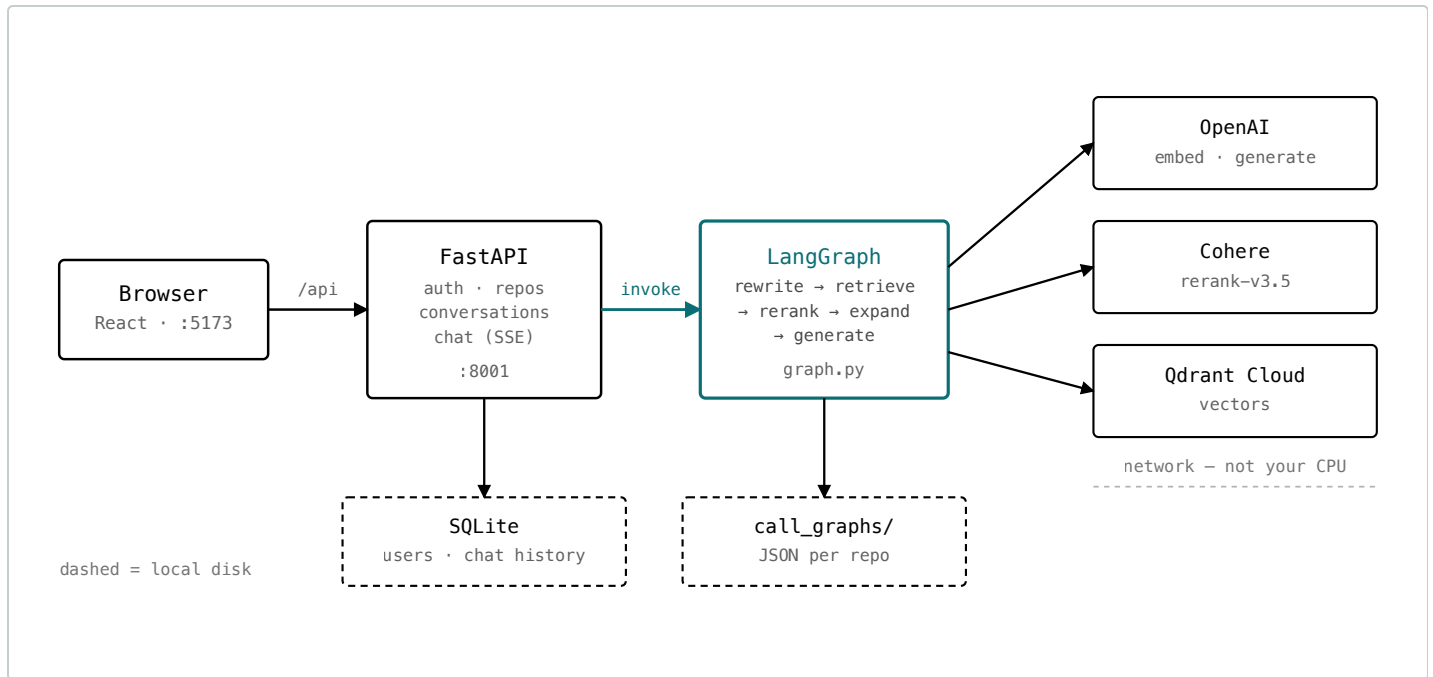
Most RAG projects are “chat with your PDF” with a vector database bolted on. The three things that make this one different are all defensible engineering choices: **structure-aware chunking**, **hybrid retrieval**, and **refusal over hallucination**. Each is discussed in its own section below, with the alternative that was rejected.

§ 02

System architecture

Two processes you run, four services you call. Almost nothing computes locally.

The system splits into a thin web layer and a pipeline layer. The web layer handles accounts, history, and HTTP. The pipeline layer does retrieval and generation, and it was written to run standalone from the command line before the web UI existed — which is why `ingestion.py` and `graph.py` can still be driven directly without a server.



Where work actually happens. The teal path is the pipeline. Everything to the right of it is a network call; the two dashed boxes are the only things written to local disk — and both become a problem in deployment (§14).

Folder by folder

The root holds the pipeline — these four files are the actual product, and each runs standalone from the command line with no web server involved. Everything else wraps them.

PATH	WHAT LIVES THERE	RESPONSIBILITY
<code>ingestion.py</code>	Root · pipeline	Clone, filter, chunk with tree-sitter, embed, upsert to Qdrant, build the call graph
<code>graph.py</code>	Root · pipeline	The LangGraph pipeline — all five nodes and the routing between them
<code>dense_embeddings.py</code>	Root · provider	OpenAI embedding provider, batched and concurrent
<code>reranker.py</code>	Root · provider	Cohere rerank provider, interface-compatible with the local cross-encoder
<code>semantic_chunking.py</code>	Root · provider	Alternative embedding-based chunker — present, switched off
<code>eval.py</code>	Root · quality	DeepEval harness: faithfulness, relevancy, recall
<code>backend/</code>	Web layer	<code>main.py</code> routes · <code>auth.py</code> hashing and JWTs · <code>db.py</code> engine and session · <code>models.py</code> the four tables · <code>schemas.py</code> request validation · <code>pipeline_store.py</code> shared Qdrant client and call-graph files
<code>frontend/</code>	Web layer	<code>src/pages/</code> the four routes · <code>src/components/</code> chat bubbles, citations, sidebar, auth guard · <code>src/lib/</code> API client and auth context
<code>call_graphs/</code>	Generated data	One JSON file per ingested repo — who calls whom
<code>qdrant_storage/</code>	Generated data	Local vector store. Dormant when running against Qdrant Cloud
<code>mindai.db</code>	Generated data	SQLite — users, repos, conversations, messages
<code>.env</code>	Configuration	Every secret and switch. Never committed

Why there is a `.env` file at all

Three separate problems, one solution.

- **Secrets must not be in source code.** An API key committed to git is public the moment the repo is pushed, and stays in history even after deletion. `.env` is listed in `.gitignore`, so keys live on the machine and never in the repository.

- **Settings differ per environment.** Your laptop, a teammate's laptop, and production each need different databases and endpoints. Reading them from the environment means the same code runs everywhere with no edits.
- **Reconfiguring should not require a code change.** Moving the vector store to the cloud or switching the reranker is one line in this file — no deploy of new code.

The file is loaded at startup by `load_dotenv()` , which copies its contents into the process environment. Order matters: it runs *before* any project module is imported, because several modules read their configuration at import time and would otherwise see nothing.

What each key does

KEY	USED BY	WHAT IT CONTROLS
<code>OPENAI_API_KEY</code>	Embeddings + generation	Required. Pays for dense vectors, query rewriting, and answer generation. Nothing works without it
<code>MINDAI_GENERATION_MODEL</code>	generate node	Which model writes the answer. Currently <code>gpt-4o-mini</code> — cheap, and the hard work was already done by retrieval
<code>MINDAI_REWRITE_MODEL</code>	rewrite node	Which model rewrites the question for retrieval. A small model is fine for this
<code>MINDAI_JUDGE_MODEL</code>	<code>eval.py</code> only	The grader in the eval harness — <code>gpt-4o</code> , a stronger model, because a weak judge produces meaningless scores. Never used at request time
<code>MINDAI_JWT_SECRET</code>	<code>backend/auth.py</code>	Required. Signs login tokens. Anyone holding this can forge any user's session. Changing it logs everyone out
<code>DATABASE_URL</code>	<code>backend/db.py</code>	Where relational data lives. <code>sqlite:///./mindai.db</code> today; a Postgres URL here is the entire production migration
<code>QDRANT_URL</code>	Vector store	A switch. Set → cloud mode over HTTP, concurrent. Unset → local on-disk mode, serialised behind a lock
<code>QDRANT_API_KEY</code>	Vector store	Authenticates to Qdrant Cloud. Ignored by an unauthenticated local container
<code>MINDAI_QDRANT_PATH</code>	Vector store	Where local vectors are written. Only consulted when <code>QDRANT_URL</code> is unset
<code>MINDAI_COHERE_API_KEY</code>	rerank node	A switch. Present → hosted Cohere reranking. Absent → falls back to the local cross-encoder, which is far slower on CPU
<code>MINDAI_CHUNK_STRATEGY</code>	<code>ingestion.py</code>	A switch. <code>ast</code> → syntactic boundaries. <code>semantic</code> → embedding-derived boundaries. Changing it requires re-ingesting
<code>MINDAI_CALL_GRAPH_DIR</code>	<code>pipeline_store.py</code>	Where call-graph JSON is written

`MINDAI_LLM_BASE_URL`

Optional, disabled

Points generation at any OpenAI-compatible endpoint — Gemini, a local model. Commented out after a free-tier quota ran out

THE DISTINCTION WORTH MAKING IN AN INTERVIEW

Most of these keys are *values* — a model name, a file path. Three are **switches**:

`QDRANT_URL` , `MINDAI_COHERE_API_KEY` , and `MINDAI_CHUNK_STRATEGY` . Their mere presence changes which implementation the code selects at startup, because each sits behind a provider interface with two implementations. That is what makes local-versus-cloud a configuration decision rather than a rewrite.

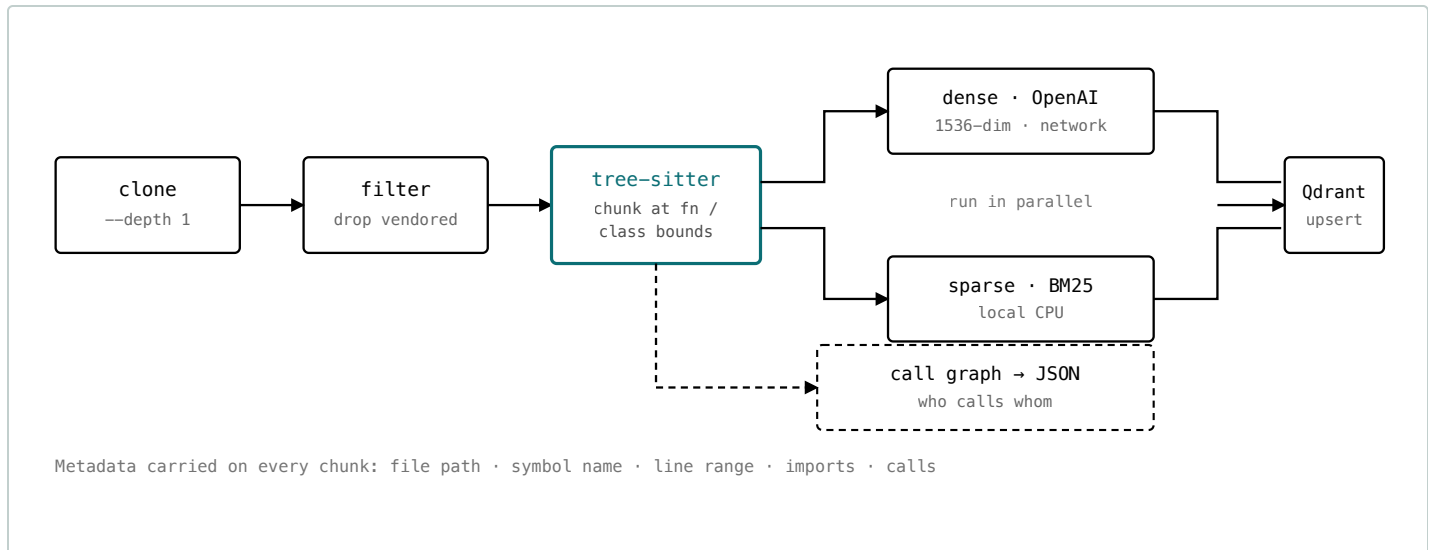
MISSING FROM THIS PROJECT

There is no `.env.example` — a committed template listing every key with placeholder values, so a new developer knows what to fill in. The README instructs you to copy one, but the file was deleted. Anyone cloning this repository cannot follow its own setup instructions.

§ 03

Ingestion pipeline

Turning a GitHub URL into a searchable index. Runs once per repository, in the background.



One pass, two indexes, one side product. Dense and sparse embedding run on separate threads because one waits on the network and the other uses the CPU — genuinely independent work, so the local pass costs nothing on the clock.

Step by step

1. **Shallow clone.** `git clone --depth 1 --single-branch` into a temp directory, deleted in a `finally` block so a failure never leaks disk.
2. **Filter noise.** Drop `node_modules`, lock files, binaries, and anything over 1 MB — almost always generated or vendored, and pure cost to parse.
3. **Chunk with tree-sitter.** Parse each file into a syntax tree and emit one chunk per function, class, method, or interface. Files in unsupported languages fall back to overlapping line windows so nothing is silently dropped.
4. **Attach metadata.** Each chunk carries its file path, symbol name, symbol type, language, exact line range, the identifiers it calls, and the imports of its file.
5. **Cap size.** Anything above 6,000 tokens is split in half recursively — a single class can otherwise exceed the embedding API's input limit and fail the whole batch.
6. **Embed twice.** Dense vectors from OpenAI, sparse BM25 vectors locally, in parallel.
7. **Upsert** into a Qdrant collection named for that repo, and write the call graph to JSON.

WHY AST CHUNKING INSTEAD OF FIXED TOKEN WINDOWS

Fixed windows cut wherever the counter runs out — usually mid-function. You get a chunk that starts halfway through one function and ends halfway through the next, so the retrieved text answers nothing completely and the line range points at a fragment.

Cutting at syntactic boundaries means a retrieved chunk is a *whole* function: it has a name, a signature, a body, and an exact line range you can cite. It is also why the call graph is possible at all — you cannot say “this function calls that one” if your chunks are not functions.

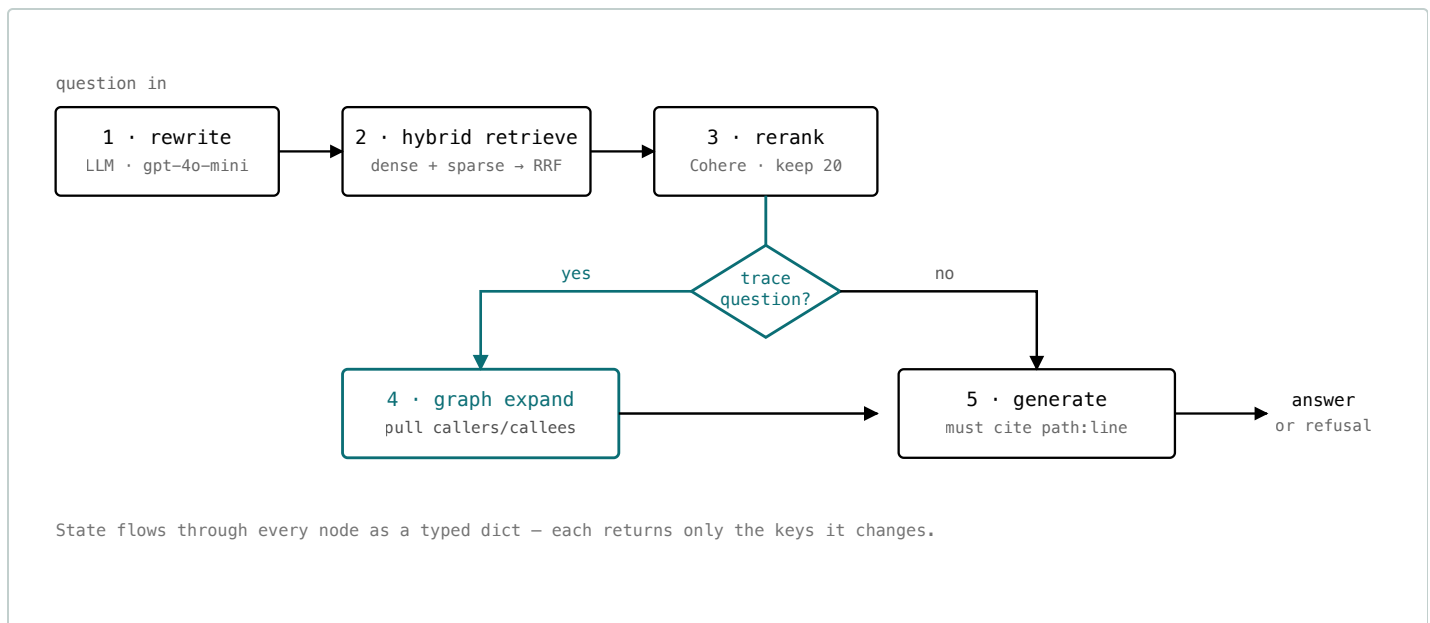
WHY IMPORTS ARE ATTACHED AT FILE LEVEL

A function's own line range cannot contain its imports — they sit at the top of the file, outside the chunk. Without them a retrieved function gives no signal about what it depends on. Attaching the file's imports to every chunk from that file restores that context.

§ 04

Query pipeline

Five nodes in a LangGraph state machine, with one conditional branch.



The only branch in the graph. Expansion is conditional because pulling in callers and callees helps a “how does this flow” question and actively hurts a “what does this one function do” question by burying the answer in unrelated code.

What each node does

NODE	DOES	RUNS ON
query_rewrite	Turns conversational phrasing into a retrieval-friendly query. "How does login work?" becomes terms likelier to match code.	OpenAI
hybrid_retrieve	Embeds the query dense and sparse, runs both searches, fuses with RRF	OpenAI + local + Qdrant
rerank	Scores every candidate against the query with a cross-encoder, keeps the top 20, records a confidence	Cohere
graph_expand	Conditional. Adds callers and callees of the retrieved symbols	Local + Qdrant
generate	Writes the answer from the retrieved chunks, with mandatory citations	OpenAI

WHY LANGGRAPH RATHER THAN A CHAIN OR PLAIN FUNCTIONS

The pipeline is not a straight line — it branches at rerank. A linear chain cannot express that without an `if` buried inside a step, which hides control flow. LangGraph makes the branch a declared edge, so the shape of the pipeline is readable from the graph definition rather than reconstructed by reading every function. Typed state also means each node declares exactly which keys it reads and writes.

§ 05

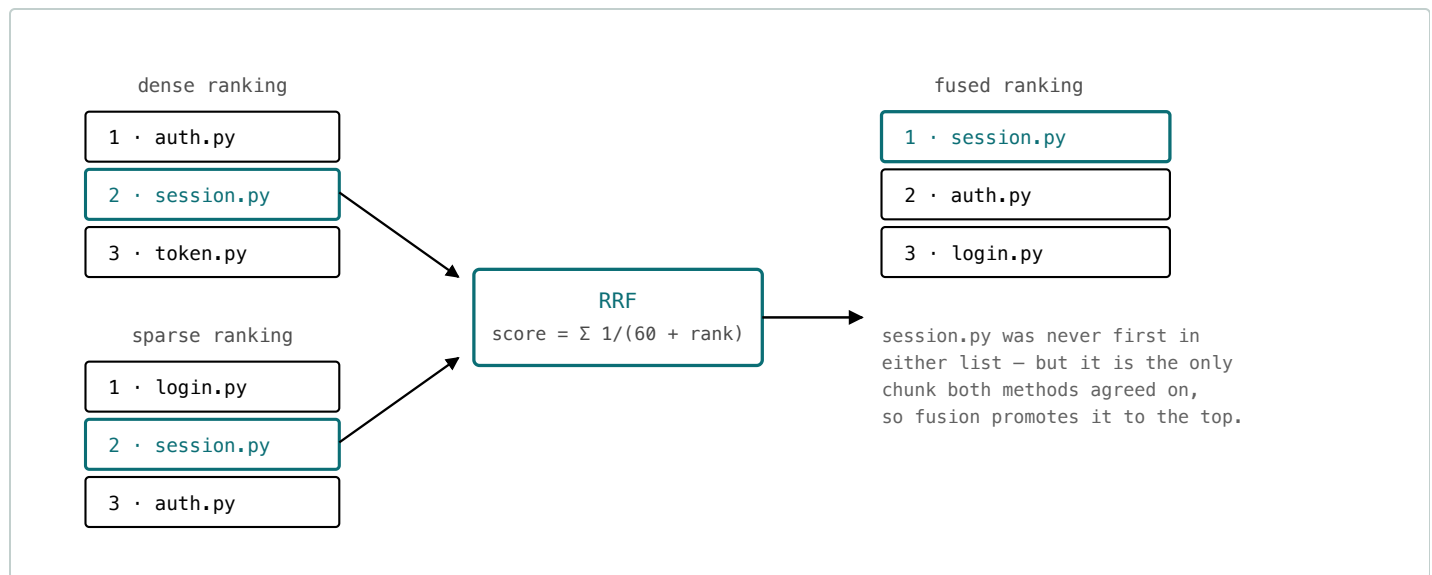
Hybrid retrieval and RRF

Two searches with opposite strengths, combined by rank rather than by score.

Dense (semantic) search converts text into a 1536-dimension vector where similar *meanings* land near each other. It finds the authentication code when you ask about “login,” even if the word “login” never appears. It is bad at exact tokens: search `UserServiceImpl` and it may return a dozen plausible-looking service classes.

Sparse (BM25) search is classic keyword matching with term weighting. It nails exact identifiers, error strings, and rare names. It is useless when your words differ from the code's words.

Code search needs both, because developers ask both kinds of question — “where do we handle rate limiting” (semantic) and “what uses `MAX_RETRIES`” (lexical).



Why fusion beats picking a winner. Agreement between two independent methods is a stronger signal than a high score from either one alone.

WHY RRF INSTEAD OF ADDING THE SCORES TOGETHER

Dense similarity and BM25 produce numbers on completely different scales — cosine similarity sits between 0 and 1, BM25 is unbounded and depends on corpus statistics. Adding or averaging them means one silently dominates, and the balance shifts with every repo you ingest.

Reciprocal Rank Fusion throws the scores away and uses only *position*: each result contributes $1 / (60 + \text{rank})$. Ranks are comparable across any two systems, so no tuning is needed and no normalisation can drift. The constant 60 damps the difference between rank 1 and rank 2 so a single confident system cannot dominate the fusion.

§ 06

Reranking

Retrieval is fast and approximate. Reranking is slow and accurate. Use each where it belongs.

Vector search compares the query vector to chunk vectors that were computed *independently*, before the query existed. That is what makes it fast enough to search millions of chunks — and what makes it imprecise, because the chunk's vector was never influenced by your question.

A cross-encoder reads the query and one chunk *together* and scores how well that chunk answers that query. Far more accurate, and far too slow to run over a whole index. So the pipeline retrieves a generous candidate set cheaply, then reranks only those candidates and keeps the top 20.

WHY THE SCORE IS KEPT, NOT JUST THE ORDER

The reranker's top score becomes the pipeline's **confidence**. If the best chunk it could find still scores poorly, that is direct evidence the index does not contain the answer — which is what triggers a refusal instead of a guess. Reranking therefore does two jobs: ordering, and knowing when to stop.

WHY COHERE'S HOSTED RERANKER RATHER THAN A LOCAL CROSS-ENCODER

The project originally ran `sentence-transformers` locally. On CPU that became the single largest per-question cost. Moving it to a hosted API replaced local compute with a network round trip — the same trade already made for embeddings. The local path still exists behind the same `.predict(pairs)` interface, so removing the API key restores it with no code change.

§ 07

Graph expansion — the “Graph” in Graph-RAG

Plain RAG retrieves chunks that *look like* the question. Code questions often need chunks that are *connected to* those chunks.

Ask “what happens when a user uploads a file?” and pure similarity search returns the upload handler. But the answer spans the handler, the validator it calls, the storage function that calls, and the error path around it. Those functions may not resemble the question at all — they are relevant because of their *relationship* to the chunk that does.

During ingestion, every chunk's body is scanned for identifiers in call position, producing a map of which symbol calls which. At query time, if the question looks like a trace question, the pipeline follows those edges from the retrieved chunks and pulls in their callers and callees before generating.

WHY REGEX EXTRACTION RATHER THAN A FULL CALL-GRAPH ANALYSER

Proper call-graph construction needs type resolution and import following — per language, and genuinely hard for dynamic languages. This project indexes 15 languages, so that cost is multiplied by 15. A regex scan for identifiers in call position is approximate: it catches real calls, and also catches some things that are not calls.

That trade is acceptable because the graph is used to *widen* a candidate set that is then reranked. A false edge pulls in an irrelevant chunk, the reranker scores it low, and it never reaches the model. A missed edge costs a little recall. Neither produces a wrong answer — which is why an approximate graph is good enough here and would not be for, say, a refactoring tool.

§ 08

Citations and refusal

The feature that separates this from a chatbot with a vector database attached.

Every chunk carries its exact file path and line range through the entire pipeline, and the generation prompt requires the model to cite `path:line` for each claim. The frontend renders those citations as clickable references, so a reader can verify any sentence against the actual source.

The refusal path is the harder engineering. Two independent conditions produce “I don't know” rather than an answer:

- **Low retrieval confidence** — the reranker's best score is below threshold, so the index probably does not contain the answer.
- **Missing citation** — the model produced prose without pointing at a source, which means it fell back on general knowledge instead of the provided context.

Refusals are stored as a flag on the message, so a refusal survives a page reload instead of appearing later as if it were a normal answer.

WHY REFUSING IS A FEATURE, NOT A LIMITATION

The purpose of the tool is to save a developer from reading the whole codebase. A wrong answer *costs* them time — they act on it, it fails, and they debug both the code and the bad explanation. A refusal costs them nothing beyond the question. So the system is deliberately tuned to prefer silence over confidence, which is the

opposite of how a general assistant is tuned.

§ 09

Tech stack, and what was rejected

Every row here is an interview question waiting to happen. The third column is the answer.

PIPELINE

TECHNOLOGY	JOB	CHOSEN OVER
LangGraph	Pipeline orchestration as a state machine	LangChain chains — cannot express the conditional branch cleanly; raw functions — control flow becomes invisible
tree-sitter	Parse 15 languages into syntax trees for chunking	Fixed token windows — cut mid-function; regex parsing — breaks on real syntax; per-language parsers — 15 dependencies
Qdrant	Vector store with native sparse-vector support	Pinecone — hosted only; Chroma — weaker hybrid support; pgvector — no native sparse vectors
FastEmbed (BM25)	Sparse lexical vectors	Elasticsearch — a whole second service for one feature
OpenAI embeddings	Dense vectors, 1536-dim	Local FastEmbed models — minutes of CPU per repo; the tradeoff is documented in the code
Cohere rerank	Cross-encoder scoring	Local sentence-transformers — dominated per-question latency on CPU
tiktoken	Token counting for the chunk budget	Character heuristics — wrong often enough to fail embedding batches
DeepEval	Retrieval and answer quality metrics	Eyeballing outputs — not defensible in review

TECHNOLOGY	JOB	CHOSEN OVER
FastAPI	HTTP API, background ingestion tasks, SSE streaming	Flask — no native async or typed validation; Django — far more framework than four tables need
SQLAlchemy	ORM over users, repos, conversations, messages	Raw SQL — no portability to Postgres later
bcrypt + JWT	Password hashing and stateless sessions	Auth0/Clerk — external dependency for a portfolio project; plain sessions — needs server-side state
React + Vite	Chat UI and dev server with API proxy	Next.js — SSR buys nothing for an authenticated single-page chat
Tailwind	Styling	Component libraries — heavier than a four-page app needs
SSE	Streaming pipeline progress to the browser	WebSockets — bidirectional, unnecessary here; polling — worse latency and more requests

THE PATTERN WORTH POINTING OUT

Three components are swappable through a single environment variable, each behind an identical interface: vector store (local or cloud), reranker (local or hosted), chunker (syntactic or semantic). That is why moving the vector store to the cloud was a one-function change rather than a refactor. If an interviewer asks about extensibility, this is the concrete example.

Data stores

Three stores, deliberately separated by what they are good at.

STORE	HOLDS	WHY THIS ONE
Qdrant	Chunk vectors and payloads — one collection per repo	Vector similarity is not something a relational database does well; per-repo collections give isolation for free
SQLite	Users, repo metadata, conversations, messages	Relational data with real foreign keys; swaps to Postgres by changing one URL
JSON files	Call graph per collection	A nested map that is read whole and written whole — a table would add schema for no query benefit

The link between the first two is one column: the repo row stores its Qdrant collection name. That single field is what ties relational data to vector data, and it is why moving the vector store leaves old repo rows pointing at collections that no longer exist.

KNOWN WEAKNESS

Both SQLite and the call-graph JSON live on local disk. In a container that disk is erased on every restart. See §14.

Backend and frontend

API surface

ROUTE	PURPOSE
POST /auth/signup · /auth/login	Create account, return a JWT
GET /auth/me	Current user from the bearer token
POST /repos	Start ingestion; returns immediately with status <code>pending</code>
GET /repos	List repos and their ingestion status
POST /conversations	Start a conversation against a repo
GET /conversations/{id}/messages	Full history for reload
POST /chat	Ask a question; streams progress and the answer over SSE

WHY INGESTION RETURNS BEFORE IT FINISHES

Ingesting a repo takes minutes. Holding an HTTP connection open that long fails on every proxy and timeout in the path. Instead the row is created as `pending`, the response is sent, and the work runs as a background task that moves the row through `ingesting` to `ready` or `failed`. The UI polls the status.

Frontend

Four routes — landing, login, signup, chat — with a guard component that redirects unauthenticated users. The JWT lives in `localStorage`. Chat uses raw `fetch` with a stream reader rather than the browser's `EventSource`, because `EventSource` cannot send an `Authorization` header — a small detail that shapes the whole client.

§ 12

Evaluation

If you cannot measure a RAG system, you are guessing about every change you make.

METRIC	QUESTION IT ANSWERS	CATCHES
Faithfulness	Is every claim supported by the retrieved context?	Hallucination — the model padding with general knowledge
Answer relevancy	Does the answer address what was asked?	Correct but off-topic responses
Contextual recall	Did retrieval surface the chunks needed to answer?	Retrieval failures, as distinct from generation failures

WHY THESE THREE SPECIFICALLY

They separate the two halves of the system. Contextual recall grades *retrieval*; faithfulness and relevancy grade *generation*. When quality drops, that split tells you which half to fix — without it you are tuning blind, changing chunk sizes to fix what was actually a prompt problem.

Measured performance

Real numbers from the running system, ingesting a 69-file repository into 414 chunks.

0.08s All local CPU work, whole repo	2.93s git clone	0.04s tree-sitter chunking, 414 chunks
0.17s bcrypt, per login	95 MB Backend memory, idle	0 Neural inference run locally

The headline result is the last one. No model runs on the machine hosting this application — embeddings, reranking, and generation are all network calls. BM25 is lexical statistics, not a neural model, and runs no inference.

The consequence worth stating out loud: **password hashing is the most expensive computation in the entire project**, costing roughly twice the local CPU of ingesting a whole repository. That is by design — bcrypt is deliberately slow — but it means this system needs no GPU and fits comfortably in a small container, which is unusual for anything labelled RAG.

Deployment readiness

What works as-is, and the three things that break on a cloud host.

Because all inference is remote, the deployment target is small and cheap — no GPU, modest memory. What does not survive the move is anything assuming a permanent local filesystem.

1 · SQLITE IS ERASED ON EVERY RESTART

Containers have ephemeral disks. Every deploy or crash wipes the database — all accounts and all chat history. The fix needs no code change: point `DATABASE_URL` at managed Postgres. The database layer already branches on the URL scheme.

2 · CALL GRAPHS VANISH SILENTLY

Same ephemeral disk, but this failure is invisible. Graph expansion simply finds nothing and stops expanding — answers get worse on trace questions with no error anywhere. Fix: move the call graph into the database or object storage.

3 · INGESTION CAN BE KILLED MID-RUN

Ingestion is a background task lasting minutes. If the platform recycles or sleeps the container during it, the repo is stranded as `ingesting` forever with nothing to retry it. The container also needs `git` installed, which slim base images omit.

ALSO REQUIRED

- Serve the built frontend from FastAPI so one process handles API and UI
- Add a health endpoint — every platform needs one for readiness checks
- Bake the BM25 vocabulary into the image so cold starts do not depend on a model download
- Drop the unused torch dependency — 389 MB locally, several GB on Linux where it pulls CUDA wheels by default

§ 15

Questions you will be asked

q Why not just use a bigger context window and paste the whole repo in?

Cost and accuracy. A large repo exceeds even long context windows, and models reliably lose detail in the middle of very long inputs. Retrieval gives the model a small, high-relevance context — cheaper per query and more accurate. It also makes citation possible: you know which chunks were used because you chose them.

q How do you know retrieval is actually working?

Contextual recall in the DeepEval harness grades retrieval separately from generation. Reranker confidence is also logged per query, so a pattern of low-confidence answers is visible without reading outputs.

q What happens when the repository changes?

Currently re-ingestion rebuilds the collection. The honest answer is that incremental updates are not implemented — the design would key chunks by content hash and re-embed only changed files, which the chunk-ID scheme already anticipates.

Q Why two embedding types instead of one good one?

They fail in opposite directions. Dense search misses exact identifiers; sparse search misses paraphrases. Developers ask both kinds of question, and fusing by rank rather than score means neither method needs tuning against the other.

Q What is the weakest part of this system?

Two things. The call graph is regex-derived, so it is approximate — acceptable because it only widens a candidate set that gets reranked, but it would not support a refactoring tool. And chunk-level retrieval cannot answer questions whose answer is genuinely spread across many files; graph expansion mitigates that but does not solve it.

Q How would you scale this to a thousand users?

Postgres instead of SQLite, call graphs out of local files, and ingestion moved from in-process background tasks to a real queue with retries. The vector store is already managed and horizontally scalable. Because no inference is local, the web tier scales by adding small stateless containers.

Q Why does it refuse to answer sometimes?

Deliberately. Low reranker confidence or a missing citation triggers a refusal. For a tool whose job is to save reading time, a confident wrong answer costs the user more than no answer.
